

Amazon S3 Server-Side Encryption with Customer-Provided Keys (SSE-C)

To Begin with the Lab

Summary of the Lab

In this lab, **Amazon S3 Server-Side Encryption with Customer-Provided Keys (SSE-C)** was demonstrated using the AWS CLI. Unlike SSE-S3 and SSE-KMS, the encryption key is generated, managed, and supplied entirely by the customer, while AWS never stores the key. The lab generated a 256-bit encryption key and its MD5 hash using OpenSSL, configured AWS CLI credentials, uploaded an object to S3 using SSE-C, and verified that the object was encrypted. Finally, the same key was provided while downloading the object, proving that the data could only be decrypted with the original customer-provided key.

Understanding S3 Server-Side Encryption with Customer-Provided Keys (SSE-C)

SSE-C (Server-Side Encryption with Customer-Provided Keys) allows customers to encrypt S3 objects using **their own encryption keys** instead of keys managed by AWS. During upload, the client provides the encryption key to Amazon S3, which uses it to encrypt the object and immediately discards the key after encryption. During download, the exact same key must be supplied again so that Amazon S3 can decrypt the object before returning it. Since AWS never stores the encryption key, customers have complete ownership and responsibility for key management.

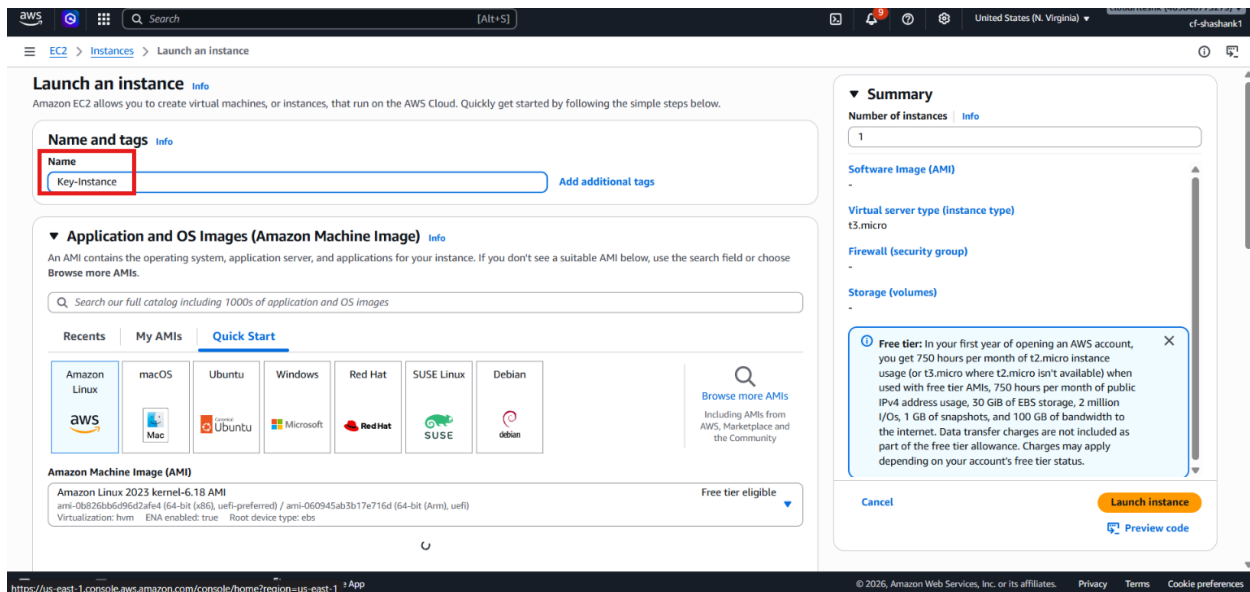
Example:

A company has strict security regulations requiring complete ownership of encryption keys. Instead of allowing AWS to manage keys through SSE-S3 or KMS, the company generates its own AES-256 key using OpenSSL, uploads confidential files using SSE-C, and stores the key securely in its own environment. Without that original key, nobody—including AWS—can decrypt the stored object.

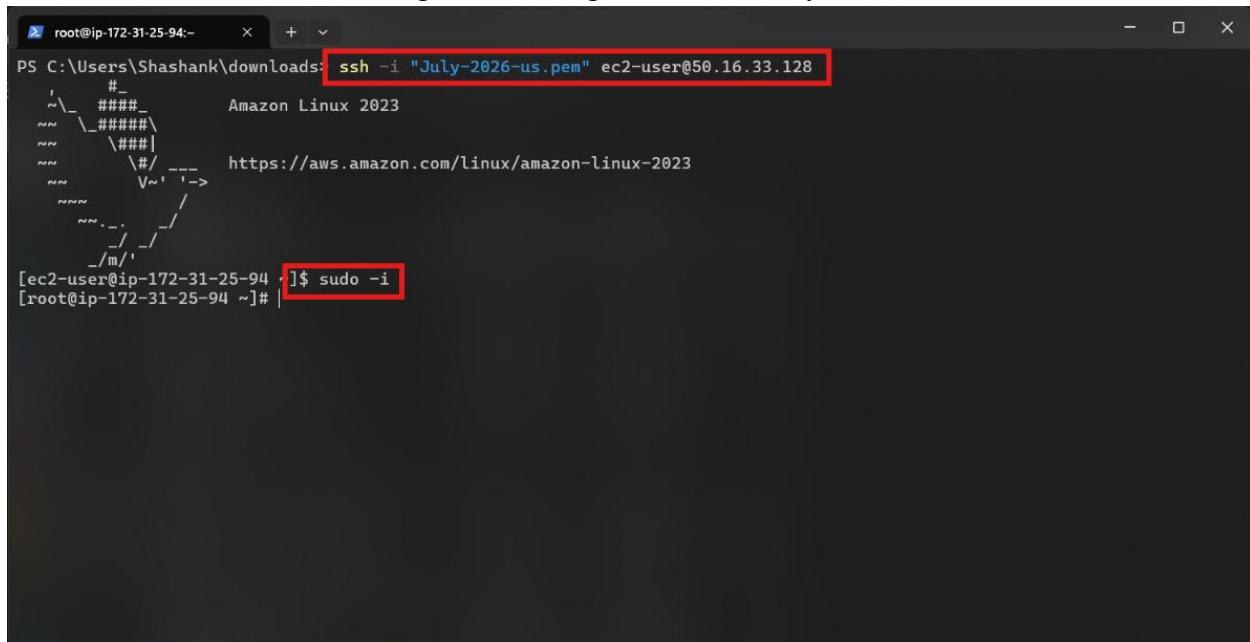
Lab Steps

Step 1 – Generate the customer encryption key

- Launch an **Amazon Linux EC2** instance.



- Connect to the instance using SSH and login as root user by “sudo -i” command.



- Verify that **OpenSSL** is installed.
- Generate a **256-bit AES encryption key** by running this command.

```
openssl rand 32 > sse-c.key
```

- Convert the key into **Base64** format.

```
key=$(cat sse-c.key | base64)
```

- Store the generated key in a file.
- Generate the corresponding **MD5 hash** required by AWS CLI.

```
keymd5=$(cat sse-c.key | openssl dgst -md5 -binary | base64)
```

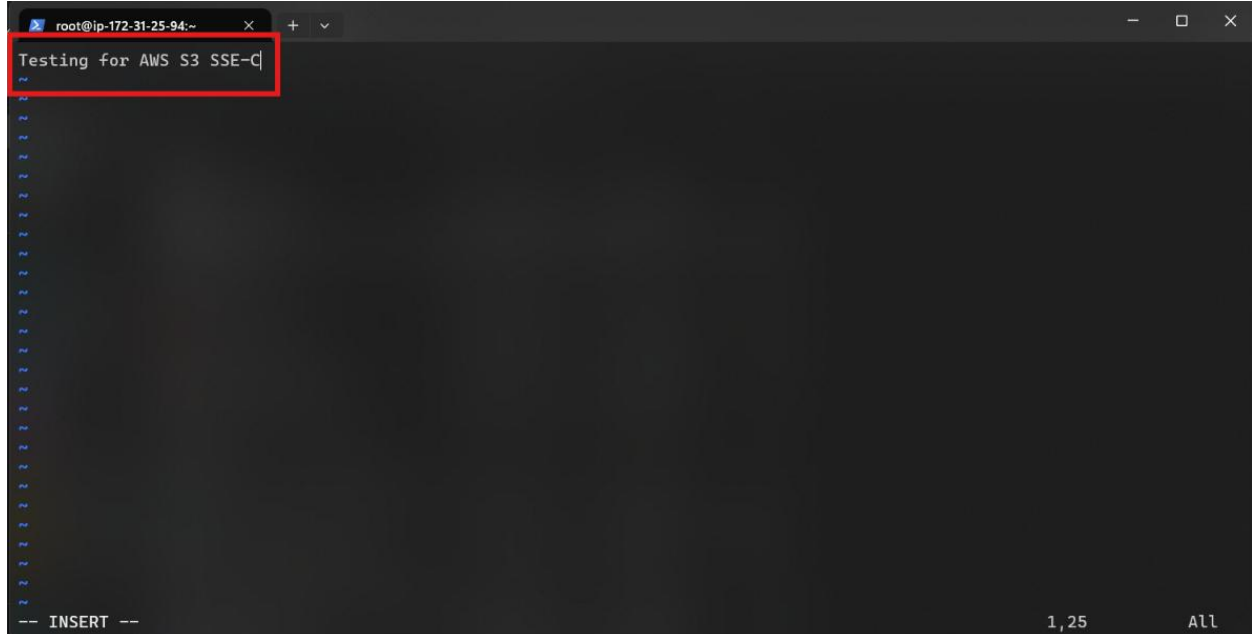
- Store both the Base64 key and MD5 hash in environment variables.

Step 3 – Prepare the upload file

- Create a simple text file.

```
vi <FILE_NAME>
```

- Add sample content to the file: Testing for AWS S3 SSE-C.



```
root@ip-172-31-25-94:~  
Testing for AWS S3 SSE-C  
-- INSERT -- 1,25 All
```

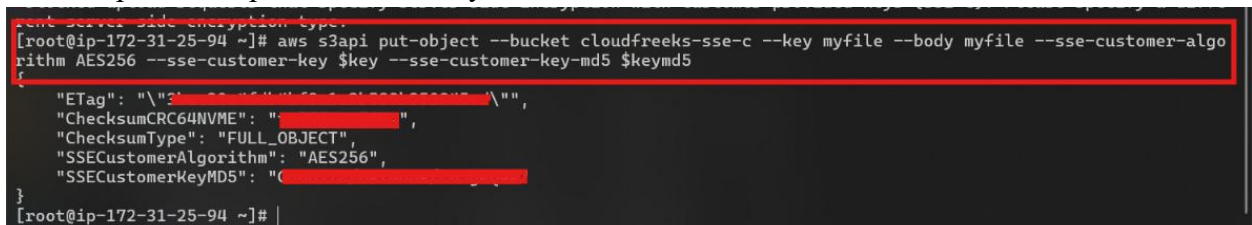
- Save the file locally on the EC2 instance.

Step 4 – Upload the object using SSE-C

- Use the AWS CLI put-object command.

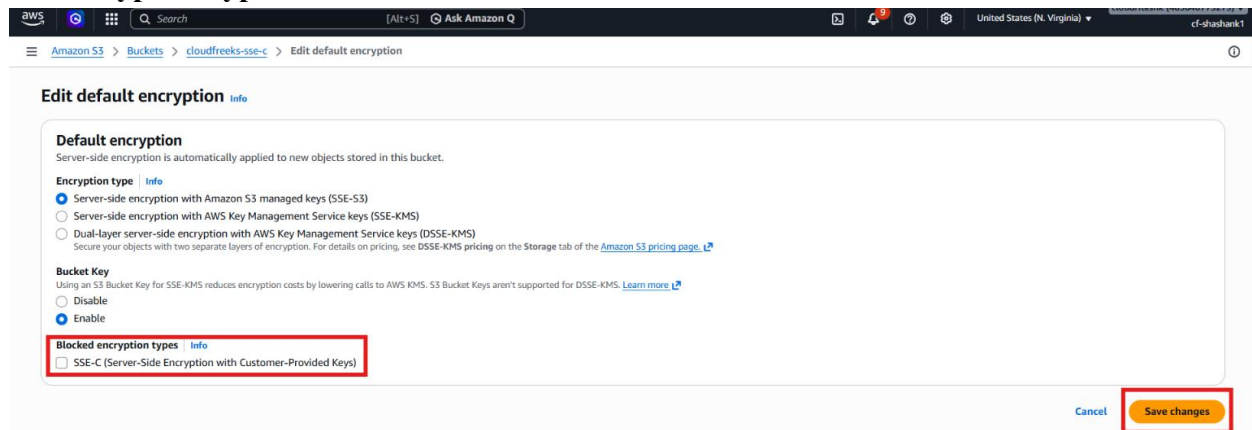
```
aws s3api put-object --bucket <BUCKET_NAME> --key <FILE_NAME> --body  
<FILE_NAME> --sse-customer-algorithm AES256 --sse-customer-key $key --sse-customer-key-  
md5 $keymd5
```

- Specify the S3 bucket name.
- Specify the object name.
- Upload the file.
- Select **AES256** as the customer encryption algorithm.
- Provide the Base64 encryption key.
- Provide the MD5 hash of the key.
- Complete the upload successfully.



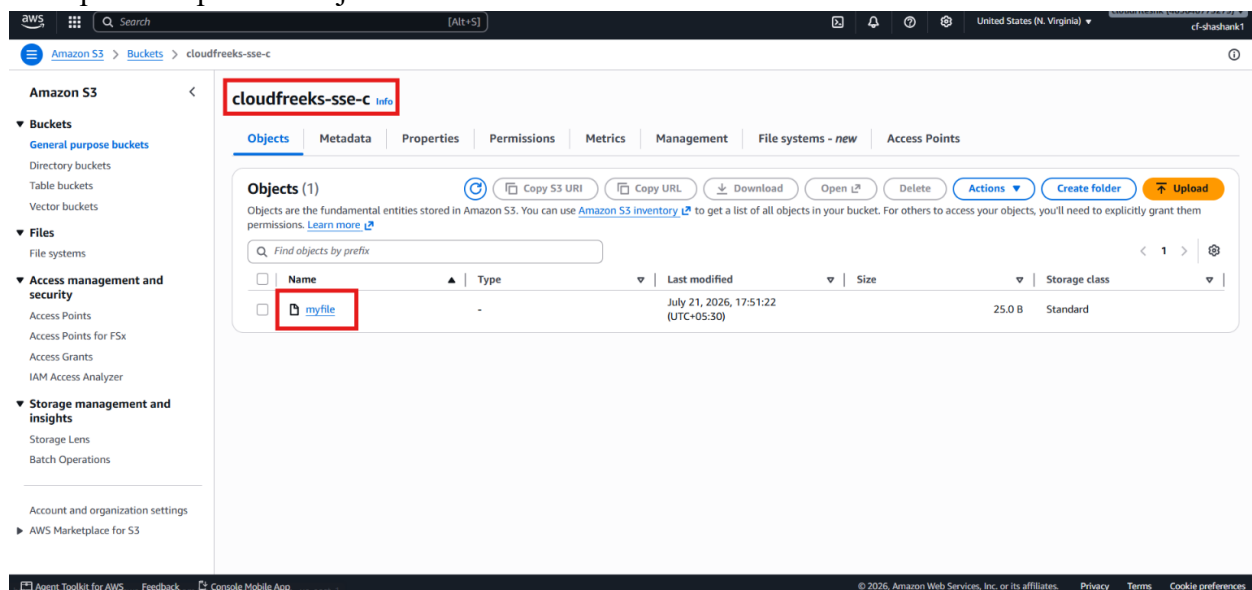
```
root@ip-172-31-25-94:~# aws s3api put-object --bucket cloudfreeks-sse-c --key myfile --body myfile --sse-customer-algo-  
rithm AES256 --sse-customer-key $key --sse-customer-key-md5 $keymd5  
{  
  "ETag": "\"31234567890123456789012345678901\"",  
  "ChecksumCRC64NVME": "\"31234567890123456789012345678901\"",  
  "ChecksumType": "FULL_OBJECT",  
  "SSECustomerAlgorithm": "AES256",  
  "SSECustomerKeyMD5": "\"31234567890123456789012345678901\"",  
}  
[root@ip-172-31-25-94 ~]#
```

- If you cannot upload file, make sure your bucket has disabled SSE-C under **Blocked encryption types**.

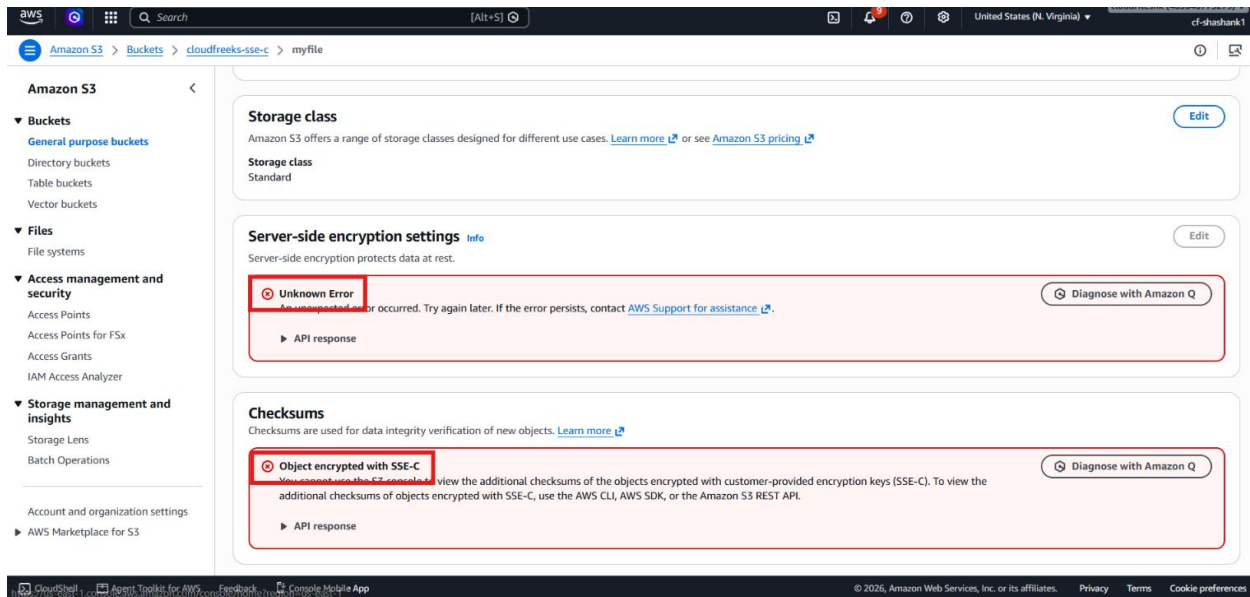


Step 5 – Verify the encrypted object

- Open the uploaded object in the S3 console.



- Navigate to the object properties.
- Verify that the object is encrypted using SSE-C.



- Observe that the AWS Console cannot display or download the object because there is no way to supply the customer-provided key through the GUI.

Step 6 – Download the object using the customer key

- Delete the local copy of the uploaded file.

```
rm -f <FILE_NAME>
```

- Use the AWS CLI get-object command.
- Provide the same AES256 algorithm.
- Supply the original Base64 encryption key.
- Supply the corresponding MD5 hash.
- Download the file using the command.

```
aws s3api get-object <FILE_NAME> --bucket <BUCKET_NAME> --key <FILE_NAME> --  
sse-customer-algorithm AES256 --sse-customer-key $key --sse-customer-key-md5 $keymd5
```

- Download the object successfully.
- Open the file and verify that the original contents are restored correctly.

```
root@ip-172-31-25-94:~  
[root@ip-172-31-25-94 ~]# rm -f myfile  
[root@ip-172-31-25-94 ~]# ls  
sse-c.key  
[root@ip-172-31-25-94 ~]# aws s3api get-object <FILE_NAME> --bucket <BUCKET_NAME> --key <FILE_NAME> --sse-customer-algor  
ithm AES256 --sse-customer-key $key --sse-customer-key-md5 $keymd5  
-bash: FILE_NAME: No such file or directory  
[root@ip-172-31-25-94 ~]# aws s3api get-object myfile --bucket cloudfreeks-sse-c --key myfile --sse-customer-algorithm A  
ES256 --sse-customer-key $key --sse-customer-key-md5 $keymd5  
{  
  "AcceptRanges": "bytes",  
  "LastModified": "2026-07-21T12:21:22+00:00",  
  "ContentLength": 25,  
  "ETag": "\"[REDACTED]\"",  
  "ChecksumCRC64NVME": "[REDACTED]",  
  "ChecksumType": "FULL_OBJECT",  
  "ContentType": "binary/octet-stream",  
  "Metadata": {},  
  "SSECustomerAlgorithm": "AES256",  
  "SSECustomerKeyMD5": "[REDACTED]"  
}  
[root@ip-172-31-25-94 ~]# ls  
myfile sse-c.key  
[root@ip-172-31-25-94 ~]# cat myfile  
Testing for AWS S3 SSE-C  
[root@ip-172-31-25-94 ~]# |
```

Verification

- Verified that a 256-bit customer-managed encryption key was generated using OpenSSL.
- Verified that both the Base64 key and MD5 hash were created successfully.
- Verified that AWS CLI was configured before performing SSE-C operations.
- Verified that the object was uploaded using **SSE-C** encryption.
- Verified that the S3 Console recognized the object as **SSE-C encrypted**.
- Verified that the encrypted object could not be accessed through the AWS Console.
- Verified that supplying the original customer-provided key during download successfully decrypted the object.
- Verified that without the original key, the encrypted object cannot be decrypted or accessed.